

3. Робота з файлами: щоденний цикл Git

Повний цикл на прикладі

1. Створи файл:

```
echo "print('Hello Git!')" > main.py
```

2. Подивись, що бачить Git:

```
git status
# Untracked files:
#   main.py
```

3. Додай до stage:

```
git add main.py
```

4. Зафіксуй зміни (коміт):

```
git commit -m "add main.py"
```

Коміт — це «знімок» стану проекту. У кожного коміту є унікальний хеш, автор, дата і повідомлення:

```
git log
# commit 3d4e1339a1... (HEAD -> main)
# Author: Igor <igor@example.com>
# Date:   Tue Aug 18 12:37:00 2026 +0300
#
#   add main.py
```

Основні команди

Команда	Опис
<code>git status</code>	показує зміни у файлах
<code>git add <file></code>	додає файл до stage
<code>git commit -m "..."</code>	створює коміт
<code>git log --oneline</code>	коротка історія комітів
<code>git diff</code>	показує відмінності
<code>git restore <file></code>	скасовує локальні зміни
<code>git rm <file></code>	видаляє файл з репозиторію

Деталі, які знадобляться щодня

`git add`

```
git add main.py      # один файл
git add .            # усі зміни в поточній теці й нижче
git add *.py         # усі python-файли
git add -A           # усі зміни в усьому репозиторії
```

`git add .` зручний, але небезпечний: легко закомітити зайве. Перед комітом завжди дивись `git status`.

`git commit`

```
git commit -m "add login form"      # звичайний коміт
git commit -am "fix typo"           # add + commit для вже відслідковуваних файлів
git commit --amend -m "нове повідомлення" # переписати ОСТАННІЙ коміт
```

Як писати повідомлення:

- коротко, до 50 символів, у наказовому способі: `add user login`, `fix division by zero`;
- одна логічна зміна = один коміт;
- погано: `правки`, `фікс`, `asdasd`, `остаточно тепер точно`.

`--amend` не застосовуй до комітів, які вже запущені: він змінює хеш, а значить переписує історію, яку інші вже завантажили.

git log

```
git log          # повна історія
git log --oneline # по одному рядку на коміт
git log --oneline --graph --all # з деревом гілок
git log -3       # останні три коміти
git log --stat   # плюс перелік змінених файлів
git show <хеш>  # що саме змінилося в конкретному коміті
```

git diff

```
git diff          # working directory vs staging — «що я ще не додав»
git diff --staged # staging vs останній коміт — «що піде в коміт»
git diff main.py  # різниця по одному файлу
```

Читати вивід: `-` — рядок був, `+` — рядок став.

Скасування змін

```
git restore main.py          # відкотити правки у файлі (незакомічене ЗНИКНЕ)
git restore --staged main.py # прибрати зі stage, правки залишити
git restore .                 # відкотити все — обережно!
```

Старий синтаксис, який зустрінете в тьюторіалах: `git checkout -- main.py` і `git reset HEAD main.py`. `git restore` — сучасна заміна, робить те саме зрозуміліше.

Видалення та перейменування

```
git rm old.py          # видалити файл і одразу підготувати зміну
git rm --cached secret.env # прибрати з Git, але залишити на диску
git mv old.py new.py    # перейменувати
```

Стани файлу в Git

```
untracked → staged → committed
  (git add)   (git commit)
```

- **untracked** — Git бачить файл, але не відслідковує його;
- **modified** — відслідковуваний файл змінено, але не додано до stage;
- **staged** — зміну підготовлено до коміту;
- **committed** — зміна в історії.

`git status` завжди підказує наступний крок — читай його вивід, а не вгадуй.

